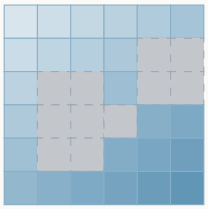


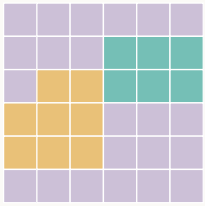
(T, H, W, C)

UNE VIDÉO



$x \longrightarrow E_\theta(x)$

ON EN CACHE



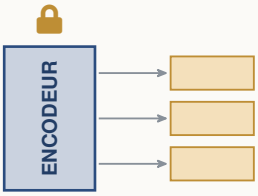
$\|\hat{z} - z\|_1$

ON DEVINE L'IDÉE

V-JEPA 2.1 pas à pas

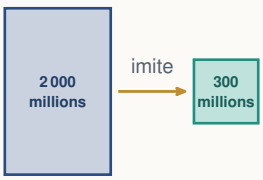
Comprendre, coder et déployer un modèle vidéo auto-supervisé, en partant vraiment de zéro

Aucun prérequis : on part vraiment de zéro.



une sonde par tâche

ENCODEUR GELÉ



presque la même qualité

DISTILLATION

Doctor Mokira

2026

Édition 2026. Document composé avec LaTeX (pdfTeX), polices Palatino (texte et mathématiques), TeX Gyre Heros (titres) et DejaVu Sans Mono (code). Figures et schémas réalisés avec TikZ et pgfplots, coloration syntaxique assurée par minted/Pygments, encadrés par tcolorbox, bibliographie par biblatex.

Ce livre explique de bout en bout l'article *V-JEPA 2.1 : Unlocking Dense Features in Video Self-Supervised Learning*, publié par l'équipe FAIR de Meta et l'Université de Saragosse. Il est écrit pour être lu sans aucun prérequis : on part de zéro, on explique avec des mots avant d'écrire des symboles, et on n'avance jamais sans avoir d'abord dit pourquoi. Il ne s'arrête pas à la compréhension : il conduit le lecteur jusqu'au code qui tourne, puis jusqu'au modèle mis en service.

Avant-propos

« Un article de recherche ne devrait effrayer personne. »

Ce livre raconte un seul article scientifique, du premier mot jusqu'à la dernière ligne de code : *V-JEPA 2.1 : Unlocking Dense Features in Video Self-Supervised Learning* [2], écrit par l'équipe FAIR de Meta et l'Université de Saragosse. Cet article présente un programme qui apprend à comprendre des vidéos tout seul, sans qu'un humain lui explique jamais ce qu'il regarde. Il fait suite à un premier modèle, V-JEPA 2 [1], dont nous parlerons beaucoup : c'est en corrigeant ses défauts que V-JEPA 2.1 est né.

Un article de recherche est écrit par des spécialistes, pour des spécialistes. Il va vite. Il emploie des mots que personne ne vous a jamais définis. Il suppose que vous connaissez déjà dix autres articles. Résultat : beaucoup de gens curieux, capables, motivés, s'arrêtent à la première page.

Ici, on ne suppose rien du tout. Chaque idée est d'abord racontée avec des mots de tous les jours, puis dessinée, puis seulement écrite en langage mathématique, puis enfin mise en pratique avec du code que vous pouvez faire tourner vous-même.

> **En clair**
>
> Notre promesse : si vous savez compter et si vous savez lire, vous pouvez comprendre comment fonctionne V-JEPA 2.1. Le reste, nous le construisons ensemble, dans l'ordre, sans jamais sauter une marche.

Pour qui est ce livre

Pour toute personne qui veut trois choses à la fois :

1. **comprendre** ce que fait V-JEPA 2.1, et pourquoi il fait mieux que le modèle qui existait avant lui ;

2. **écrire elle-même** le modèle et le programme qui l'entraîne ;
3. **mettre le modèle au travail** pour de vrai, tous les jours, au service d'une application.

Aucun diplôme n'est demandé. Vous n'avez pas besoin de savoir ce qu'est un réseau de neurones, ni un transformeur, ni même une image numérique : tout cela est expliqué à partir de rien, dans les quatre premiers chapitres.

Comment ce livre est fabriqué

Huit règles ont guidé chaque page, et elles ne sont pas des intentions : ce sont des engagements que vous pouvez vérifier.

1. Aucun mot technique n'apparaît sans avoir été défini juste avant.
2. On commence toujours par les prérequis, jamais par la nouveauté.
3. Un dessin pour chaque idée qui bouge, qui circule ou qui se compare.
4. Un exemple après chaque notion, avec des chiffres quand c'est possible.
5. Des mots simples, ceux de tous les jours.
6. Le texte vous parle et vous accompagne.
7. Une phrase de transition à la fin de chaque section, qui annonce la suite.
8. Des phrases entières, comme dans un vrai livre.

Le texte utilise quelques encadrés qui reviennent souvent, faciles à reconnaître à leur pictogramme :

- **En clair** — l'idée expliquée avec des mots de tous les jours.
- **L'intuition** — l'image mentale à garder en tête.
- **À quoi ça sert en IA ?** — où cette notion sert vraiment dans V-JEPA 2.1.
- **Sur des données réelles** — la même idée, appliquée aux chiffres réels de l'article.
- **Ce qu'il faut savoir avant** — les seuls prérequis de la section.
- **Piège à éviter** — l'erreur classique que tout le monde fait.
- **À retenir** — l'essentiel, si vous ne deviez garder qu'une chose.

- 🚀 **Pour aller plus loin** — un pont vers un sujet plus avancé, que vous pouvez sauter en première lecture.

Chaque chapitre se termine par des exercices, tous corrigés. Ils ne servent pas à vous noter : ils servent à vous montrer, à vous-même, que vous avez compris.

Un mot d'honnêteté

L'article explique comment *entraîner* le modèle et comment l'*évaluer*. Il ne parle presque pas de sa *mise en service* : pas de serveur, pas de temps de réponse, pas de surveillance. La dernière partie de ce livre sera donc de deux couleurs, et nous vous le dirons franchement à chaque fois :

- **Ce que l'article dit** — la taille des modèles, la résolution des images, la vitesse de calcul. Ce sont des résultats mesurés.
- **Ce que nous ajoutons** — le serveur, la mesure du temps de réponse, la surveillance. Ce sont des conseils de métier, utiles et éprouvés, mais qui ne viennent pas de l'article.

Nous dirons aussi, aussi clairement que ses réussites, ce que ce modèle ne sait pas faire, et les tâches sur lesquelles d'autres modèles restent meilleurs que lui. Un livre qui ne raconte que les victoires ne vous aide pas à choisir.

Bonne lecture.

Table des matières

Avant-propos iii
Table des matières vii

I Les fondations : partir vraiment de zéro 1
1 Une image, une vidéo, vues par un ordinateur 3

1.1 Un ordinateur ne voit pas des objets, il voit des nombres . . . 4

1.2 La résolution : la finesse de la mosaïque 6

1.3 La couleur, c'est trois grilles au lieu d'une 7

1.4 Ranger les nombres : le tenseur et sa forme 9

1.5 Une vidéo, c'est une pile d'images 11

1.6 Ce que tout cela pèse vraiment 14

1.7 Passons au code 16

Exercices 18

II Comprendre JEPA et V-JEPA 2, l'ancêtre direct 21

III Le problème que V-JEPA 2.1 vient résoudre 23

IV V-JEPA 2.1, ingrédient par ingrédient 25

V Les résultats, lus avec honnêteté 27

VI Construire le modèle et le programme d'entraînement 29

VII De l'entraînement à la production 31

Bibliographie 33

Première partie

Les fondations : partir vraiment de zéro

CHAPITRE 1

Une image, une vidéo, vues par un ordinateur

Avant de parler d'intelligence artificielle, de vidéos cachées et de modèles à deux milliards de réglages, mettons-nous d'accord sur une chose toute simple : qu'est-ce qu'une image, pour une machine ? Ce chapitre pose le décor, tranquillement, sans rien supposer.

> **Ce qu'il faut savoir avant**
>
> Rien du tout. Vraiment rien. Il faut savoir compter, savoir qu'une multiplication sert à répéter une addition, et c'est tout. Chaque mot nouveau sera expliqué au moment où il arrive.

Vous regardez une photo de chat. Vous voyez un chat. C'est immédiat, c'est gratuit, vous n'avez rien eu à faire. Un ordinateur, lui, ne voit pas de chat. Il ne voit même pas d'image. Il voit une très longue liste de nombres, et rien d'autre.

Tout ce livre part de là. Si nous voulons qu'une machine finisse par comprendre une vidéo, il faut d'abord savoir sous quelle forme la vidéo arrive chez elle. C'est exactement ce que nous allons faire dans ce chapitre : ouvrir le capot et regarder les nombres.

1.1 Un ordinateur ne voit pas des objets, il voit des nombres

En clair

Une image, pour un ordinateur, c'est une grille de petits carrés. Chaque carré porte un nombre, et ce nombre dit une seule chose : à quel point ce petit carré est clair ou sombre.

Pensez à une mosaïque, comme celles qu'on voit sur les murs des vieilles maisons. De loin, vous voyez un dessin. De près, vous voyez des centaines de petits carreaux collés les uns aux autres, chacun d'une seule couleur. Une image numérique, c'est exactement cela : une mosaïque de petits carrés, tous de la même taille, rangés en lignes et en colonnes.

Définition 1.1 (Le pixel)

Un *pixel* est un de ces petits carrés. C'est le plus petit morceau d'une image : on ne peut pas le couper en deux. Le mot vient de l'anglais *picture element*, c'est-à-dire « élément d'image ». Chaque pixel porte un nombre qui décrit sa couleur.

Commençons par le cas le plus simple : une image en *niveaux de gris*, c'est-à-dire une image sans couleur, comme une vieille photo en noir et blanc. Dans ce cas, un seul nombre suffit par pixel. On se met d'accord une fois pour toutes sur ce que ce nombre veut dire :

- le nombre **0** veut dire « ce carré est complètement noir » ;
- le nombre **255** veut dire « ce carré est complètement blanc » ;
- tous les nombres entre les deux sont des gris, de plus en plus clairs à mesure qu'on monte.

La figure 1.1 montre les deux faces de la même image : à gauche ce que la machine a réellement en mémoire, à droite ce que nous voyons apparaître à l'écran.

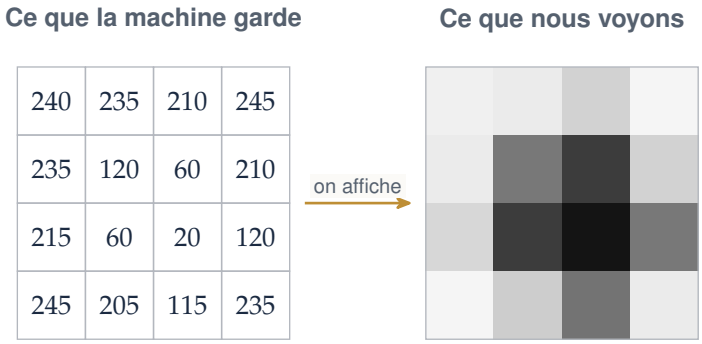


FIGURE 1.1. Les deux visages d'une même image. À gauche, les seize nombres que l'ordinateur garde en mémoire. À droite, le dessin obtenu en peignant chaque carré : petit nombre, carré sombre ; grand nombre, carré clair. Il n'y a rien d'autre dans une image que ces nombres.

Exemple 1.2 (Lire trois pixels)

Dans la grille de gauche de la figure 1.1, le pixel tout en haut à gauche vaut 240 : il est presque blanc. Le pixel du milieu vaut 20 : il est presque noir. Un pixel qui vaudrait 128, soit à peu près la moitié de 255, serait un gris moyen, ni clair ni foncé.

L'intuition

Voyez chaque pixel comme un petit interrupteur à gradateur, du genre de ceux qui règlent la lumière d'un salon. Tourné à fond dans un sens, la lampe est éteinte : c'est 0. Tourné à fond dans l'autre, la lampe éclaire au maximum : c'est 255. Une image, c'est un mur couvert de milliers de ces petites lampes, chacune réglée à sa valeur.

Piège à éviter

Pourquoi 255 et pas 100, ou 1000 ? C'est une simple convention, née de la façon dont les ordinateurs rangent les nombres : avec huit cases de mémoire, on peut écrire exactement 256 valeurs différentes, de 0 à 255. Retenez surtout le sens de lecture : **plus le nombre est grand, plus le pixel est clair**. Beaucoup de débutants inversent, dessinent des images

en négatif, et cherchent l'erreur pendant une heure.

Nous savons maintenant ce que contient une case de la mosaïque. La question suivante vient toute seule : combien y a-t-il de cases ?

1.2 La résolution : la finesse de la mosaïque

Définition 1.3 (La résolution)

La *résolution* d'une image, c'est le nombre de pixels qu'elle contient, donné sous la forme « largeur × hauteur ». Une image de résolution 256×256 possède 256 colonnes de pixels et 256 lignes de pixels.

Plus la mosaïque a de carreaux, plus le dessin est fin. Avec très peu de carreaux, un rond ressemble à un escalier ; avec beaucoup, il redevient un rond. La figure 1.2 montre le même disque dessiné deux fois, la première fois grossièrement, la seconde finement.

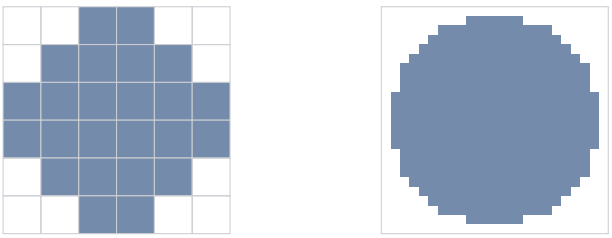


FIGURE 1.2. Le même disque, dessiné avec deux finesses différentes. À gauche, la mosaïque est grossière et les bords deviennent des marches d'escalier. À droite, les carreaux sont seize fois plus nombreux et l'œil retrouve un vrai rond. Rien n'a changé dans le dessin : seule la finesse de la grille a changé.

Le nombre total de pixels s'obtient par une simple multiplication : on multiplie la largeur par la hauteur, comme on calculerait le nombre de carreaux d'un sol.

Exemple 1.4 (Compter les pixels d'une vraie image)

Une image de résolution 256×256, qui est la taille utilisée pendant la plus grande partie de l'entraînement de V-JEPA 2.1, contient

$$256 \times 256 = 65\,536 \text{ pixels.}$$

Soit soixante-cinq mille petits carrés. Et ce n'est encore qu'une image en gris : nous n'avons même pas parlé de couleur.

À retenir

Deux images peuvent montrer exactement la même chose et ne pas avoir la même résolution. Quand la résolution monte, le dessin gagne en finesse, mais le nombre de nombres à traiter grimpe très vite : il grimpe comme le *carré* de la taille. Doubler la largeur et la hauteur, c'est multiplier le travail par quatre. Cette remarque, qui paraît anodine ici, expliquera plus tard pourquoi la dernière phase d'entraînement de V-JEPA 2.1 est si courte.

Nos images sont encore grises. Ajoutons la couleur : vous allez voir que cela ne change presque rien à l'histoire.

1.3 La couleur, c'est trois grilles au lieu d'une

Un écran ne sait fabriquer que trois couleurs de lumière : du rouge, du vert et du bleu. Toutes les autres couleurs sont obtenues en mélangeant ces trois-là, en dosant chacune plus ou moins fort. Du rouge à fond plus du vert à fond donne du jaune. Les trois à fond donnent du blanc. Les trois éteintes donnent du noir.

Un pixel en couleur a donc besoin non pas d'un nombre, mais de trois : un pour le rouge, un pour le vert, un pour le bleu. Chacun se lit comme avant, de 0 (éteint) à 255 (à fond).

Définition 1.5 (Le canal)

Un *canal* est une grille complète de nombres qui ne s'occupe que d'une seule couleur de base. Une image en couleur possède trois canaux superposés : le canal rouge, le canal vert et le canal bleu. On les appelle souvent « les canaux R, V, B », ou « RGB » à l'anglaise.

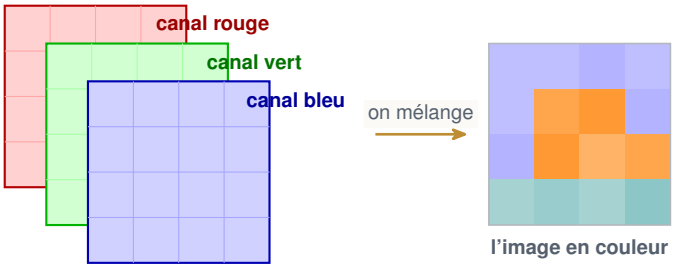


FIGURE 1.3. Une image en couleur, c'est trois mosaïques posées les unes sur les autres. La première dit combien il y a de rouge dans chaque carré, la deuxième combien de vert, la troisième combien de bleu. En regardant les trois ensemble, l'œil reconstitue la couleur.

Exemple 1.6 (Quelques couleurs, en trois nombres)

Voici comment se lisent quelques couleurs courantes, dans l'ordre rouge, vert, bleu :

Couleur	Rouge	Vert	Bleu
Noir	0	0	0
Blanc	255	255	255
Rouge vif	255	0	0
Jaune	255	255	0
Gris moyen	128	128	128

Vous remarquez que le gris s'obtient en donnant la même valeur aux trois canaux. C'est logique : aucune couleur ne tire la couverture à elle.

Comptons maintenant, pour de bon, le nombre de cases d'une image en couleur.

Exemple 1.7 (Le poids d'une image de 256×256)

Notre image possède 65 536 pixels, et chaque pixel réclame trois nombres. Au total :

$$256 \times 256 \times 3 = 196\,608 \text{ nombres.}$$

Presque deux cent mille nombres pour une seule petite image carrée, plus petite que la moitié d'un écran de téléphone. Gardez ce chiffre en tête : il va grossir très vite.

Nous avons beaucoup de nombres sur les bras. Il est temps de les ranger proprement, et de leur donner un nom.

1.4 Ranger les nombres : le tenseur et sa forme

Pour parler de tous ces nombres sans se perdre, les informaticiens utilisent un mot unique, qui a l'air savant mais qui ne cache rien de compliqué : le *tenseur*. Approchons-le par étapes, en montant une marche à la fois.

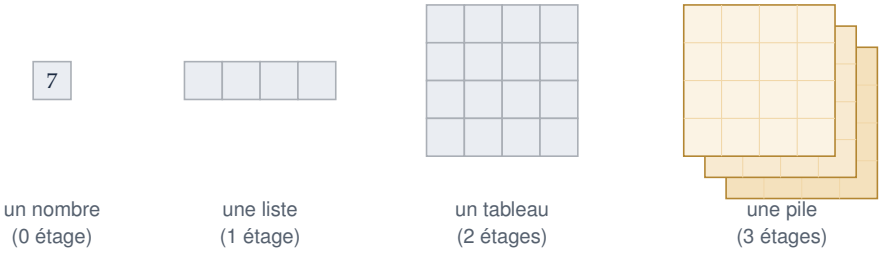


FIGURE 1.4. La même idée, à quatre étages. Un nombre tout seul ; une liste de nombres ; un tableau de nombres ; puis une pile de tableaux. Le mot « tenseur » désigne tout cela d'un coup, quel que soit le nombre d'étages.

Définition 1.8 (Le tenseur)

Un *tenseur* est un tableau de nombres qui peut avoir autant de directions que l'on veut. Un nombre seul est un tenseur sans direction ; une liste est un tenseur à une direction ; un tableau à lignes et colonnes est un tenseur à deux directions ; une pile de tableaux est un tenseur à trois directions, et ainsi de suite. Chaque direction s'appelle un *axe*.

Pour décrire un tenseur, il suffit de dire combien de cases il compte le long de chaque axe. Cette liste de tailles porte un nom, et c'est le mot le plus utile de tout le chapitre.

Notation 1.9 (La forme d'un tenseur)

La *forme* d'un tenseur est la liste du nombre de cases sur chaque axe. On l'écrit entre parenthèses. Pour une image en couleur, on écrit

$$(\text{hauteur},\ \text{largeur},\ \text{canaux}),$$

et notre image de tout à l'heure a donc la forme $(256, 256, 3)$.

Exemple 1.10 (Lire une forme)

Un tenseur de forme $(64, 384, 384, 3)$ se lit de gauche à droite comme une phrase : « soixante-quatre choses, chacune haute de 384 cases, large de 384 cases, et épaisse de 3 canaux ». Nous verrons dans un instant que ces soixante-quatre choses sont les images d'une vidéo.

Piège à éviter

L'ordre des axes n'est pas le même partout, et c'est la source d'erreur numéro un des débutants. Deux conventions se partagent le monde :

- $(\text{hauteur},\ \text{largeur},\ \text{canaux})$ — l'ordre des outils d'affichage et de photo ;
- $(\text{canaux},\ \text{hauteur},\ \text{largeur})$ — l'ordre des bibliothèques d'apprentissage comme PyTorch.

Les nombres sont exactement les mêmes, seule leur façon d'être rangés change. Mais si vous vous trompez, votre programme croira qu'une image de 256×256 en couleur est une image de 3 pixels de haut sur 256 de large avec 256 canaux, et vous obtiendrez une bouillie. Prenez l'habitude, dès aujourd'hui, d'écrire la forme attendue en commentaire au-dessus de chaque ligne de code qui manipule une image.

Nous savons ranger une image. Passons à ce qui nous intéresse vraiment dans ce livre : la vidéo.

1.5 Une vidéo, c'est une pile d'images

Une vidéo n'est rien d'autre qu'une suite d'images fixes, montrées les unes après les autres, très vite. Si elles défilent assez vite, notre œil ne voit plus des images séparées : il voit du mouvement. C'est le principe du petit carnet à feuilleter, celui où l'on dessine un bonhomme dans le coin de chaque page et où l'on fait défiler les pages avec le pouce.

Définition 1.11 (L'image d'une vidéo)

Chacune des images fixes qui composent une vidéo s'appelle une *image*, ou une *trame* ; en anglais on dit *frame*, et ce mot revient tellement souvent dans les articles et dans le code qu'il faut le connaître. Une vidéo est donc une pile d'images empilées non pas dans l'espace, mais dans le temps.

Définition 1.12 (Les images par seconde)

Le nombre d'*images par seconde* dit combien d'images sont montrées pendant une seconde. On l'abrège souvent en *fps*, de l'anglais *frames per second*. Un film de cinéma tourne à 24 images par seconde ; les vidéos de téléphone, souvent à 30 ou 60.

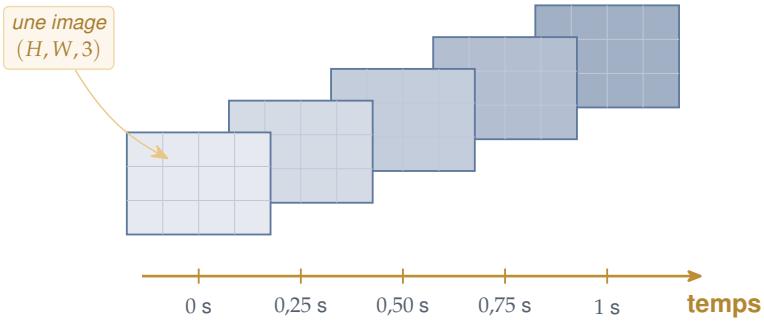


FIGURE 1.5. Une vidéo vue comme une pile de feuilles. Chaque feuille est une image complète, avec ses trois canaux de couleur. La flèche du bas est le temps : elle avance d'un cran à chaque nouvelle feuille. Ici, quatre images par seconde, donc un quart de seconde entre deux feuilles.

Puisque la vidéo est une pile d'images, sa forme s'obtient en ajoutant un axe

devant celle d'une image : l'axe du temps.

Notation 1.13 (La forme d'une vidéo)

Un extrait de vidéo s'écrit sous la forme

$$(\text{temps},\ \text{hauteur},\ \text{largeur},\ \text{canaux}),$$

où le premier nombre est le nombre d'images de l'extrait. On appelle souvent un tel extrait un *clip* : c'est un petit morceau découpé dans une vidéo plus longue, exactement comme on découpe un morceau dans une bobine de film.

La durée d'un clip se calcule sans effort. Si vous avez un certain nombre d'images, et que vous savez combien d'images passent en une seconde, alors la division vous donne les secondes :

$$\text{durée en secondes} = \frac{\text{nombre d'images}}{\text{images par seconde}}.$$

Exemple 1.14 (La durée d'un clip d'entraînement)

Pendant la plus grande partie de son entraînement, V-JEPA 2.1 regarde des clips de 16 images prises à raison de 4 images par seconde. Ces clips durent donc

$$\frac{16}{4} = 4 \text{ secondes}.$$

Quatre secondes, c'est court, mais c'est bien assez pour voir un geste commencer et se terminer : une main qui saisit une tasse, une porte qui s'ouvre. Et c'est justement ce que le modèle doit apprendre à comprendre.

Sur des données réelles

L'article de V-JEPA 2.1 utilise deux réglages, l'un après l'autre. Pendant la longue phase d'entraînement principale, les clips font 16 images en 256×256. Pendant la courte phase finale, appelée phase de refroidissement, on monte à 64 images en 384×384. Toujours à quatre images

par seconde, ces clips-là durent

$$\frac{64}{4} = 16 \text{ secondes.}$$

Le modèle regarde donc, à la fin, des extraits quatre fois plus longs et beaucoup plus fins. Nous verrons au chapitre consacré au refroidissement pourquoi cet ordre — d'abord grossier et court, ensuite fin et long — n'est pas un caprice.

Comptons enfin les nombres contenus dans un clip, en multipliant tout simplement les quatre tailles entre elles.

Exemple 1.15 (Le nombre de cases d'un clip)

Pour un clip de 16 images en 256×256 couleur :

$$16 \times 256 \times 256 \times 3 = 3\,145\,728 \text{ nombres.}$$

Plus de trois millions. Et pour un clip de la phase de refroidissement, 64 images en 384×384 :

$$64 \times 384 \times 384 \times 3 = 28\,311\,552 \text{ nombres,}$$

soit environ vingt-huit millions, c'est-à-dire neuf fois plus que le précédent.

À retenir

Une image est un tenseur à trois axes, une vidéo un tenseur à quatre axes. Rien de plus mystérieux : on a seulement ajouté le temps devant. Toutes les formes que vous croiserez dans ce livre se lisent avec cette seule règle.

Trois millions de nombres, cela reste abstrait. Regardons ce que cela représente vraiment dans la mémoire d'une machine.

1.6 Ce que tout cela pèse vraiment

Un ordinateur range ses nombres dans des cases de mémoire. L'unité de mesure de cette mémoire est l'*octet*.

Définition 1.16 (L'octet)

Un *octet* est un petit paquet de mémoire, celui qui suffit tout juste à écrire un nombre entier compris entre 0 et 255. C'est aussi l'unité dans laquelle on mesure la taille des fichiers. Mille octets font environ un kilo-octet, un million d'octets font environ un méga-octet (noté Mo), un milliard d'octets font environ un giga-octet (noté Go).

Nos pixels vont de 0 à 255 : ils tiennent donc chacun dans un seul octet. Mais dès qu'un modèle se met à calculer, il a besoin de nombres à virgule — parce qu'une moyenne, une multiplication ou une petite correction tombent rarement sur un compte rond. Ces nombres à virgule s'appellent des *nombres flottants*, et le format le plus courant en occupe quatre octets chacun, soit quatre fois plus.

Exemple 1.17 (Du fichier à la mémoire de calcul)

Notre clip de 3 145 728 nombres pèse :

- 3 145 728 octets, soit environ **3 Mo**, tant que les pixels restent des entiers de 0 à 255 ;
- $3\,145\,728 \times 4 = 12\,582\,912$ octets, soit environ **12,6 Mo**, une fois convertis en nombres à virgule pour le calcul.

Or un modèle ne regarde jamais un seul clip à la fois : il en regarde tout un paquet d'un coup, pour aller plus vite. Ce paquet s'appelle un *lot*, et sa taille est un réglage du programme d'entraînement.

Exemple 1.18 (Le poids d'un lot entier)

V-JEPA 2.1 traite 128 clips à la fois. Pour la phase principale, cela donne

$$128 \times 12\,582\,912 \approx 1\,610\,000\,000 \text{ octets},$$

soit environ **1,6 Go** — rien que pour les vidéos d'entrée, avant même le

moindre calcul. Pour la phase de refroidissement, où chaque clip pèse neuf fois plus, on arrive à environ **14,5 Go**. Voilà, en une ligne de calcul, pourquoi cette phase-là ne dure que très peu de temps : elle coûte cher.

À quoi ça sert en IA ?

Ce chapitre a l'air de ne parler que de pixels, mais il pose déjà les trois problèmes que tout le reste du livre s'emploie à résoudre.

- **Il y a beaucoup trop de nombres.** Aucun modèle ne peut traiter trois millions de valeurs une par une. Il faudra donc d'abord les regrouper en petits carrés de 16×16 pixels : c'est ce que nous ferons au chapitre 4, consacré au transformeur de vision.
- **La finesse coûte cher.** Chaque fois que la résolution monte, la facture explose. C'est pour cela que l'entraînement commence en basse résolution et ne monte qu'à la toute fin.
- **Lire une vidéo prend du temps.** Une vidéo est stockée compressée ; il faut la décompresser image par image avant de pouvoir en faire quoi que ce soit. En production, ce travail-là devient souvent plus lent que le modèle lui-même. Nous y reviendrons longuement dans la partie consacrée à la mise en service.

Pour aller plus loin

Dans le même temps où il regarde 128 clips, V-JEPA 2.1 regarde aussi 2 304 images fixes. En reprenant nos calculs, ces images pèsent $2\,304 \times 196\,608 \times 4 \approx 1{,}8$ Go, c'est-à-dire à peu près autant que les 1,6 Go de vidéo. Ces deux nombres, 128 et 2 304, n'ont donc pas été tirés au hasard : ils ont été choisis ensemble pour que les deux moitiés du travail coûtent à peu près pareil. Nous y reviendrons au chapitre sur les données.

Assez de calculs sur le papier. Vérifions tout cela sur une vraie machine.

1.7 Passons au code

Nous allons manipuler nos images et nos clips avec Python, le langage utilisé par presque tous les travaux d'intelligence artificielle, et avec `numpy`, la bibliothèque qui sait manipuler les tenseurs. Vous n'avez pas besoin de comprendre chaque ligne dès maintenant : lisez d'abord les commentaires, et comparez les nombres affichés avec ceux que nous venons de calculer à la main.

PYTHON • Fabriquer une image et lire sa forme

```python
import numpy as np

# Une image en couleur de 256 x 256 pixels, entierement noire.
# La forme se lit : (hauteur, largeur, canaux).
image = np.zeros((256, 256, 3), dtype=np.uint8)

print(image.shape)   # -> (256, 256, 3)
print(image.size)    # -> 196608   le nombre total de cases
print(image.dtype)   # -> uint8    un entier de 0 a 255
print(image.nbytes)  # -> 196608   un octet par case

# Peignons le pixel de la ligne 10, colonne 20, en rouge vif.
image[10, 20] = [255, 0, 0]
print(image[10, 20])  # -> [255   0   0]
```

Le mot `uint8` se lit « entier non signé sur huit bits » : c'est exactement le type de nombre qui va de 0 à 255, celui dont nous avons parlé. Passons maintenant à la vidéo.

PYTHON • Un clip, sa forme, sa durée et son poids

```python
import numpy as np

# Un clip de 16 images de 256 x 256 en couleur.
# La forme se lit : (temps, hauteur, largeur, canaux).
clip = np.zeros((16, 256, 256, 3), dtype=np.uint8)

print(clip.shape)   # -> (16, 256, 256, 3)
print(clip.size)    # -> 3145728   les trois millions annonces

images_par_seconde = 4.0
```

```
duree = clip.shape[0] / images_par_seconde
print(duree, "secondes")            # -> 4.0 secondes

# Le meme clip converti en nombres a virgule, ramenes entre 0 et 1.
clip_flottant = clip.astype(np.float32) / 255.0
print(clip_flottant.nbytes / 1e6, "Mo")   # -> 12.582912 Mo
```

Enfin, voici le changement d'ordre des axes dont nous parlions plus haut. C'est une opération que vous ferez des centaines de fois, alors autant la voir tout de suite.

PYTHON • Passer de l'ordre « affichage » à l'ordre « PyTorch »

```python
import numpy as np
import torch

clip = np.zeros((16, 256, 256, 3), dtype=np.uint8)   # (T, H, W, C)

x = torch.from_numpy(clip).float() / 255.0
x = x.permute(3, 0, 1, 2)                            # (C, T, H, W)

print(x.shape)   # -> torch.Size([3, 16, 256, 256])

# Aucun nombre n'a ete perdu : seul l'ordre des axes a change.
print(x.numel())  # -> 3145728
```

Piège à éviter

Remarquez la dernière ligne : le nombre de cases n'a pas bougé. Changer l'ordre des axes ne crée et ne détruit aucune donnée, cela réorganise seulement la façon de les parcourir. Si jamais ce nombre change après une telle opération, c'est que vous avez commis une erreur ailleurs — et cette vérification en une ligne vous fera gagner des heures.

À retenir

Une image est une grille de nombres ; la couleur ajoute une troisième dimension ; la vidéo en ajoute une quatrième, le temps. Tout le reste de ce livre travaille sur ces tenseurs-là, et sur rien d'autre. Quand vous serez perdu dans un chapitre plus difficile, revenez à cette question toute simple : « quelle est la forme de ce que je manipule ? » Elle remet presque toujours les idées en place.

Nous savons désormais sous quelle forme une image et une vidéo arrivent dans une machine. Mais nous n'avons encore rien appris à personne : nos tenseurs restent inertes. La question suivante est donc la bonne : qu'est-ce que cela veut dire, au juste, qu'une machine « apprend » ? C'est tout le sujet du chapitre suivant.

Exercices

Exercice 1 — Compter les cases

Combien de nombres contient une image en couleur de 512×512 pixels ? C'est la taille des images utilisées pendant la phase de refroidissement de V-JEPA 2.1.

Solution

On multiplie la hauteur, la largeur et le nombre de canaux : $512 \times 512 \times 3 = 262\,144 \times 3 = 786\,432$ nombres. C'est quatre fois plus qu'une image de 256×256, ce qui est normal puisque nous avons doublé les deux côtés.

Exercice 2 — La durée d'un clip

Un clip contient 32 images, prises à raison de 8 images par seconde. Combien de temps dure-t-il ? Et combien de temps durerait-il si les mêmes 32 images étaient prises à 4 images par seconde ?

Solution

Dans le premier cas, $32/8 = 4$ secondes. Dans le second, $32/4 = 8$ secondes. Le nombre d'images est le même, mais elles sont prélevées plus loin les unes des autres : le clip couvre donc une tranche de temps deux fois plus longue. C'est un réglage important, car il décide de ce que le modèle a le droit de voir.

Exercice 3 — Le poids d'un lot

Un lot contient 64 clips de 16 images en 256×256 couleur, convertis en nombres à virgule de quatre octets. Combien de méga-octets occupe ce lot ?

Solution

Un clip contient $16 \times 256 \times 256 \times 3 = 3\,145\,728$ nombres, donc $3\,145\,728 \times 4 = 12\,582\,912$ octets, soit environ 12,6 Mo. Pour 64 clips : $64 \times 12{,}6 \approx 806$ Mo, c'est-à-dire un peu plus de huit cents méga-octets. Ce lot fait exactement la moitié de celui de l'article, qui en contient 128, et l'on retrouve bien les 1,6 Go calculés plus haut en doublant ce résultat.

Exercice 4 — Lire une forme

Un collègue vous tend un tenseur de forme $(3, 64, 384, 384)$ et vous dit que c'est une vidéo. Que représente chaque nombre ? Quelle est la convention utilisée, et quelle serait la forme du même tenseur dans l'autre convention ?

Solution

Le 3 ne peut être que le nombre de canaux de couleur : aucune vidéo ne fait trois pixels de haut. Il s'agit donc de l'ordre (canaux, temps, hauteur, largeur), celui de PyTorch : une vidéo en couleur de 64 images en 384×384. Dans la convention d'affichage, le même tenseur s'écrirait $(64, 384, 384, 3)$. Les nombres qu'il contient sont exactement les mêmes ; seul leur rangement change.

Deuxième partie

Comprendre JEPA et V-JEPA 2, l’ancêtre direct

Troisième partie

Le problème que V-JEPA 2.1 vient résoudre

Quatrième partie

V-JEPA 2.1, ingrédient par ingrédient

Cinquième partie

Les résultats, lus avec honnêteté

Sixième partie

Construire le modèle et le programme d'entraînement

Septième partie

De l'entraînement à la production

Bibliographie

[1] M. ASSRAN et al. « V-JEPA 2 : Self-Supervised Video Models Enable Understanding, Prediction and Planning ». In : *arXiv preprint arXiv :2506.09985* (2025). URL : https://arxiv.org/abs/2506.09985.

[2] L. MUR-LABADIA et al. « V-JEPA 2.1 : Unlocking Dense Features in Video Self-Supervised Learning ». In : *arXiv preprint* (2026). FAIR at Meta et Universidad de Zaragoza. URL : https://github.com/facebookresearch/vjepa2.